

INSTITUTO MAUÁ DE TECNOLOGIA



Lógica e Pensamento Computacional

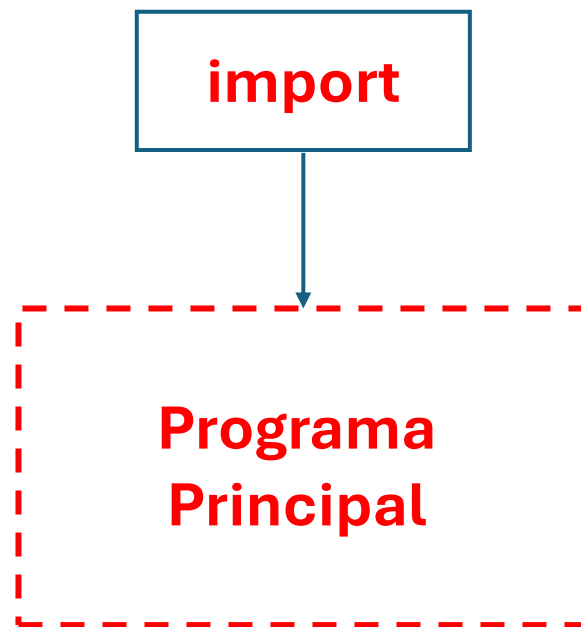
Sub-rotinas: Funções e Procedimentos

OBJETIVOS

- Revisar os conceitos das aulas anteriores
- Apresentar a estrutura de sub-rotinas (Funções e Procedimentos)
- Resolver os exercícios propostos

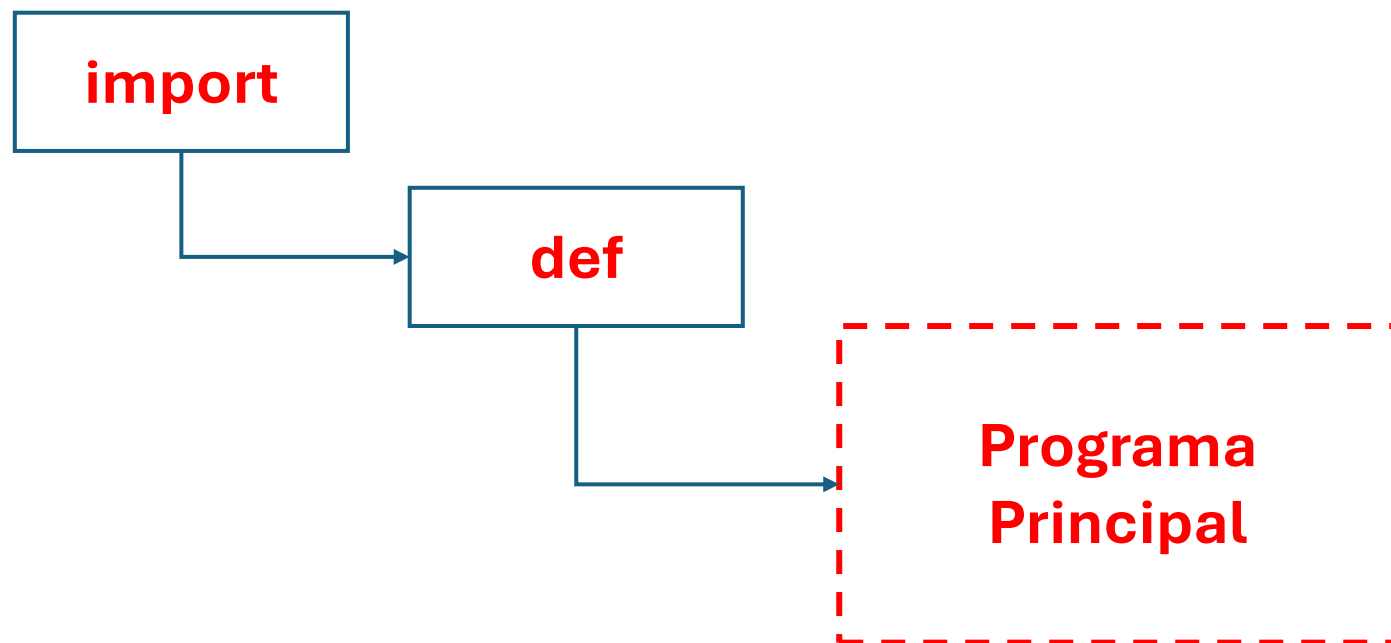
Bibliotecas: Repositório de processos já pré-programados. Normalmente as bibliotecas possuem uma temática (cálculos, gráficos, animações, sistema, etc)

Para utilizá-las no Python, vamos utilizar o comando **import**. É importante que este comando venha **antes** do programa principal, para o computador importar os comandos da biblioteca antes de utilizá-los em qualquer processo



Subrotinas: Processos pré-programados pelo próprio usuário para serem chamados em outras sub-rotinas e/ou no programa principal.

Para utilizá-las no Python, vamos utilizar o comando **def**. É importante que este comando venha **antes** do programa principal, mas **depois** do **import** das bibliotecas.

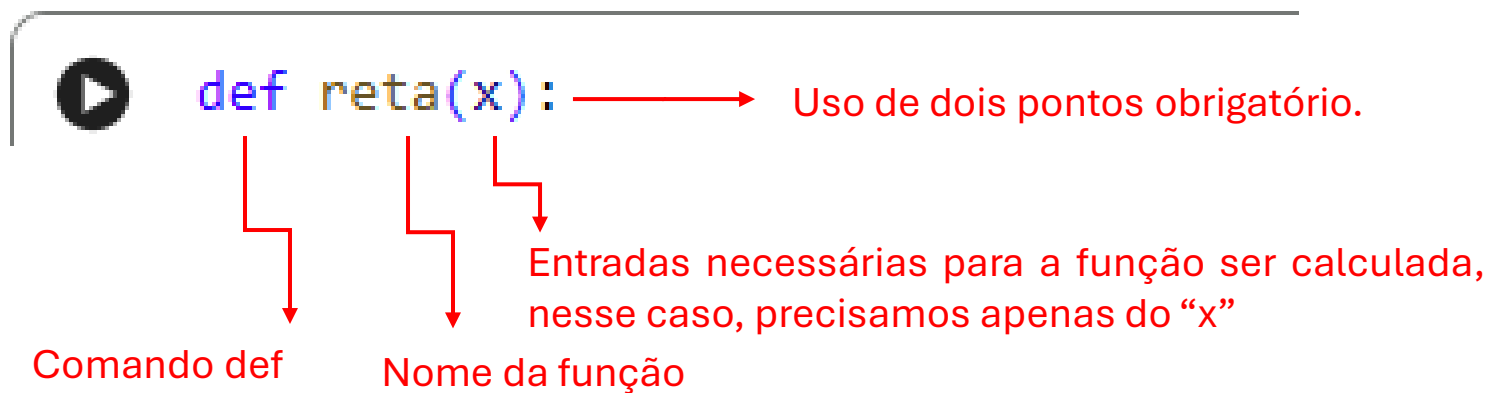


Existem dois grupos básicos de sub-rotinas:

- **Funções:** Sub-rotinas que retornam resultados para a linha onde foram “chamadas”. A função deverá ter um “nome” e os resultados deverão ser armazenados em variáveis.

Ex: Criando uma função chamada “reta” que dado x , retorna o valor de $f(x) = 2x + 3$

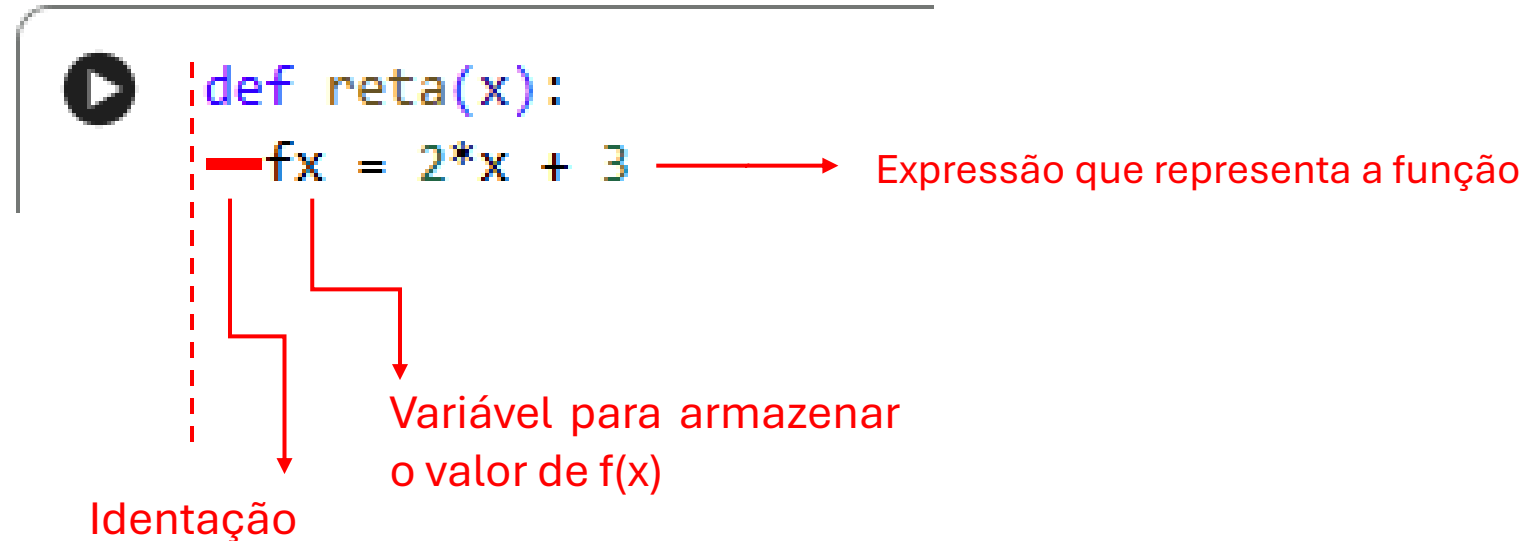
Passo 1: Criar a linha de inicialização usando **def**:



Ex: Criando uma função chamada “reta” que dado x , retorna o valor de $f(x) = 2x + 3$

Passo 2: Todos os processos que forem parte da função deverão estar **identados** em relação a linha de inicialização.

Identação: Tabulações (espaços) colocados nas linhas. Utilize a tecla “TAB” para realizar identações:

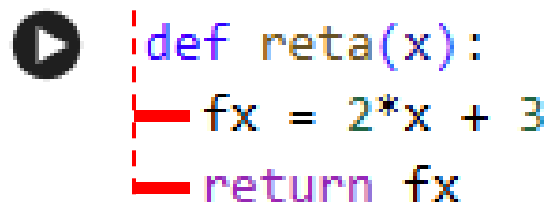


```
def reta(x):  
    fx = 2*x + 3
```

The diagram shows a code block with a play button icon. A vertical dashed line marks the start of the function body. A red arrow points from the text "Identação" to this dashed line. Another red arrow points from the text "Variável para armazenar o valor de $f(x)$ " to the variable `fx`. A third red arrow points from the text "Expressão que representa a função" to the expression `2*x + 3`.

Ex: Criando uma função chamada “reta” que dado x , retorna o valor de $f(x) = 2x + 3$

Passo 3: Após o cálculo do valor desejado, utiliza-se o comando **return** para finalizar a sub-rotina e retornar para a linha onde foi chamada:



```
def reta(x):  
    fx = 2*x + 3  
    return fx
```

Identação

Comando return para devolver o valor calculado (no caso fx) para a linha de chamada no programa principal.

Ex: Criando uma função chamada “reta” que dado x , retorna o valor de $f(x) = 2x + 3$

Passo 4: Agora logo abaixo podemos escrever o programa principal como aprendemos, mas **sem indentação**



```
def reta(x):  
    fx = 2*x + 3  
    return fx
```

Linhas indentadas que pertencem a sub-rotina “reta”

```
X = float(input("Digite o valor de X: "))
```

Programa principal iniciado com a entrada de dados. Observe a que linha do `float(input)` não está indentada, o que mostra que ela não pertence a sub-rotina

Ex: Criando uma função chamada “reta” que dado x , retorna o valor de $f(x) = 2x + 3$

DICA: Se visualmente você ficar um pouco perdido, pode utilizar linhas de **comentários** para separação. Basta inicializar a linha com o hashtag (“#”) que o computador ignorar essa linha no processamento:

```
#  
#Subrotinas  
def reta(x):  
    fx = 2*x + 3  
    return fx  
  
#  
#Programa Principal  
X = float(input("Digite o valor de X: "))
```

→ Linhas iniciadas com “#” não são lidas pelo computador. Tem a utilidade de ajudar o programador com referências, organizações ou dicas para consultas futuras

Ex: Criando uma função chamada “reta” que dado x , retorna o valor de $f(x) = 2x + 3$

Passo 5: Agora vamos “chamar” a sub-rotina no programa principal. Para isso, temos que criar uma variável e armazenar o valor que será passado no **return**:

```
# _____  
#Subrotinas  
def reta(x):  
    fx = 2*x + 3  
    return fx  
  
# _____  
#Programa Principal  
X = float(input("Digite o valor de X: "))  
  
Resultado = reta(X) _____  
  
print("O valor da função no ponto %.2f é %.2f"%(X,Resultado))
```

Criou-se uma variável Resultado que armazena o valor proveniente do return da sub-rotina. A função é chamada pelo nome da sua linha de inicialização e dentro do parênteses são colocadas as entradas necessárias. Nesse caso, a função necessitava de 1 entrada.

Testando...

```
#  
#Subrotinas  
def reta(x):  
    fx = 2*x + 3  
    return fx  
  
#  
#Programa Principal  
X = float(input("Digite o valor de X: "))  
  
Resultado = reta(X)  
  
print("O valor da função no ponto %.2f é %.2f"%(X,Resultado))  
  
... Digite o valor de X: 5  
   O valor da função no ponto 5.00 é 13.00
```

Importante: Os nomes das variáveis utilizadas no programa principal e na sub-rotina não precisam ser iguais. Basta seguir a ordem de inicialização.

Vantagem 1: Capacidade de repetição (basta chamar o nome da função quantas vezes necessitar):

```
# _____  
#Subrotinas  
def reta(x):  
    fx = 2*x + 3  
    return fx  
  
# _____  
#Programa Principal  
X1 = float(input("Digite o valor de X1: "))  
X2 = float(input("Digite o valor de X2: "))  
X3 = float(input("Digite o valor de X3: "))  
  
Resultado1 = reta(X1)  
Resultado2 = reta(X2)  
Resultado3 = reta(X3)  
  
print("O valor da função no ponto %.2f é %.2f"%(X1,Resultado1))  
print("O valor da função no ponto %.2f é %.2f"%(X2,Resultado2))  
print("O valor da função no ponto %.2f é %.2f"%(X3,Resultado3))  
  
... Digite o valor de X1: 5  
    Digite o valor de X2: 4.867  
    Digite o valor de X3: 10.12  
    O valor da função no ponto 5.00 é 13.00  
    O valor da função no ponto 4.87 é 12.73  
    O valor da função no ponto 10.12 é 23.24
```

Vantagem 2: Se modificarmos os processos da sub-rotina, o programa principal não precisa ser mudado:

O programa principal permaneceu o mesmo!

```
#
#Subrotinas
def reta(x):
    fx = -5*x + 7
    return fx

#
#Programa Principal
X1 = float(input("Digite o valor de X1: "))
X2 = float(input("Digite o valor de X2: "))
X3 = float(input("Digite o valor de X3: "))

Resultado1 = reta(X1)
Resultado2 = reta(X2)
Resultado3 = reta(X3)

print("O valor da função no ponto %.2f é %.2f"%(X1,Resultado1))
print("O valor da função no ponto %.2f é %.2f"%(X2,Resultado2))
print("O valor da função no ponto %.2f é %.2f"%(X3,Resultado3))

... Digite o valor de X1: 5
    Digite o valor de X2: 4.867
    Digite o valor de X3: 10.12
    O valor da função no ponto 5.00 é -18.00
    O valor da função no ponto 4.87 é -17.34
    O valor da função no ponto 10.12 é -43.60
```

A equação da reta foi alterada

Exemplo 1. Na aula passada fizemos um código de crescimento populacional em cidades utilizando a fórmula: $P(t) = P_0 \cdot e^{rt}$. Modifique o código, transformando os processos para se obter P(t) em uma função chamada “pop_final” colocando todas as entradas necessárias.



```
#Bibliotecas
```

```
import math
```

```
#
```

```
#Subrotinas
```

```
def pop_final(Po,r,t):
```

```
    rp = r/100
```

```
    P = Po*math.exp(rp*t)
```

```
    return P
```

Quando a função demandar mais de uma entrada, as variáveis devem ser separadas por vírgulas

```
#
```

```
#Programa Principal
```

```
Po = int(input("Digite a população atual (em habitantes): "))
```

```
r = float(input("Digite a taxa de crescimento anual (em %): "))
```

```
t = float(input("Digite o tempo (em anos): "))
```

```
P = pop_final(Po,r,t)
```

Ao chamar a função, siga a ordem das variáveis da inicialização

```
print("Após %.1f anos, a população será de %.0f habitantes"%(t,P))
```

Exemplo 2. O exemplo 3 da aula passada solicitava o cálculo da altura máxima e do alcance de um lançamento oblíquo, dado a velocidade de lançamento e o ângulo em graus. Modifique o código criando a função “obliquos” que retorna os valores de H e A. Altere o programa inicial para o usuário entrar com 3 lançamentos e apresente os resultados.

$$H = \frac{V_0^2 \cdot \sin^2(\theta)}{2g} \text{ e } A = \frac{V_0^2 \cdot \sin(2\theta)}{g}$$

```
#Bibliotecas
import math

#
#Subrotinas
def obliquos(Vo,theta):
    g = 9.81
    H = ((Vo**2)*(math.sin(math.radians(theta)))**2)/(2*g)
    A = ((Vo**2)*(math.sin(math.radians(2*theta))))/(g)
    return H,A

#
#Programa Principal
V1 = float(input("Digite a velocidade do lançamento 1 (m/s): "))
ang1 = float(input("Digite o ângulo do lançamento 1 (graus): "))
V2 = float(input("Digite a velocidade do lançamento 2 (m/s): "))
ang2 = float(input("Digite o ângulo do lançamento 2 (graus): "))
V3 = float(input("Digite a velocidade do lançamento 3 (m/s): "))
ang3 = float(input("Digite o ângulo do lançamento 3 (graus): "))

H1,A1 = obliquos(V1,ang1)
H2,A2 = obliquos(V2,ang2)
H3,A3 = obliquos(V3,ang3)

print("O Lançamento 1 teve altura máxima de %.2f m e alcance máxima de %.2f m"%(H1,A1))
print("O Lançamento 2 teve altura máxima de %.2f m e alcance máxima de %.2f m"%(H2,A2))
print("O Lançamento 3 teve altura máxima de %.2f m e alcance máxima de %.2f m"%(H3,A3))
```

Quando necessitar retornar mais de uma variável, separe as mesmas com vírgulas

Na linha de chamada da função, respeite a ordem das saídas colocadas no return.

Existem dois grupos básicos de sub-rotinas:

- **Procedimentos:** Sub-rotinas que não retornam variáveis para a linha onde foi chamada. Normalmente são utilizados para “funções do sistema” (abrir arquivos, salvar arquivos) ou para criar rotinas de **print()** para cabeçalhos ou apresentação de resultados

Sua configuração é a mesma de funções com o comando **def** mas não possui o comando **return**

Exemplo 3. Pegue o exemplo 1 desta aula e crie uma função com o nome de **entrada** para pegar os valores digitados pelo usuário e um procedimento chamado **saída** para exibir os resultados dos 3 lançamentos.

DICA: Tente compactar ainda mais o código criando uma variável **n** no procedimento que recebe o número do lançamento.

```
#Bibliotecas
import math

#
#Subrotinas
def obliquo(Vo,theta):
    g = 9.81
    H = ((Vo**2)*math.sin(math.radians(theta))**2)/(2*g)
    A = ((Vo**2)*math.sin(math.radians(2*theta)))/(g)
    return H,A

def entrada(n):
    V = float(input("Digite a velocidade do lançamento %i (m/s): "%n))
    ang = float(input("Digite o ângulo do lançamento %i (graus): "%n))
    return V,ang

def saida (n,H,A):
    print("O lançamento %i teve altura máxima de %.2f m e alcance máximo de %.2f m "%(n,H,A))

#
#Programa Principal
V1,ang1 = entrada(1)
V2,ang2 = entrada(2)
V3,ang3 = entrada(3)

H1,A1 = obliquo(V1,ang1)
H2,A2 = obliquo(V2,ang2)
H3,A3 = obliquo(V3,ang3)

saida(1,H1,A1)
saida(2,H2,A2)
saida(3,H3,A3)
```

Função para escrever as entradas

Procedimento para escrever a resposta

Observe o nível de compactação do programa principal! Iremos no futuro compactar ainda mais!